Data Communication and Computer Network Laboratory

Master of Computer Application
Second Year, Second Semester
Session: 2026-27

Assignment - I

Date: 10/08/2025

TCP Client Server Communication

1. **String Processing Server**: Implement a TCP client server application where the client sends a string to the server. The server determines the length of the string, number of vowels, number of consonants, and number of digits, and sends the results back to the client.
2. **Simple File Transfer**: Implement a TCP client server application in which the client requests a file by specifying its filename. If the file exists, the server sends its contents to the client, which stores them in a new file. If the requested file does not exist, the server sends an appropriate error response.
3. **Command Based TCP Service**: Design a TCP server supporting the following requests from a client: `TIME` ⇒ return server time; `DATE` ⇒ return server date; `HOSTNAME` ⇒ return server hostname; `ECHO <message>` ⇒ return the supplied message; and `QUIT` ⇒ terminate the connection. Define a simple application layer protocol for the requests and responses. Implement the server so that it can serve multiple clients sequentially. After one client disconnects, the server should return to `accept()` and wait for another client. Record and display the IP address and port number of every connected client.
4. **Concurrent TCP Server**: Extend the above problem statement, so that multiple clients can be served concurrently. Implement the server using either `fork()` or `POSIX` threads. Test the server by simultaneously running at least three client processes.

UDP Client-Server Communication

1. **Date and Time Server**: Implement a UDP server that responds to client requests such as `DATE`, `TIME`, and `DATETIME`. The server sends the corresponding information back to the requesting client.
2. **File Content Request Using UDP**: Write a UDP client that sends a filename to the server. If the file exists, the server sends its contents to the client. For simplicity, initially restrict the file size so that the entire content can fit within a single datagram. The server should return an error message if the file is unavailable.
3. **Chunk Based UDP File Transfer**: Extend the previous assignment to transfer files larger than a single datagram. Divide the file into fixed-size blocks and attach a sequence number to every block. The client should reconstruct the file in the correct order.
4. **Simple UDP Name Service**: Create a small server side table containing mappings such as:

   server1 → 192.168.1.10
   server2 → 192.168.1.20
   ...

   The client sends a host name, and the UDP server returns the corresponding IP address. Return "NOT FOUND" when no matching entry exists.